

Assignment 5: Exploration Strategies and Offline Reinforcement Learning

Due Monday, November 27, 11:59 pm

1 Analysis

In this section, we will analyze how reward bonuses can be used to manage distribution shift in offline RL. We consider an MDP $M = (\mathcal{S}, \mathcal{A}, r, p)$ with offline data $\mathcal{D}$ collected by a policy π_β . We denote by p_π the distribution over states induced by π .

As a motivating example, consider the soft actor-critic (SAC) algorithm discussed in HW 3. When updating, SAC adds an adjustment of $b(\mathbf{s}, \mathbf{a}) = -\log \pi(\mathbf{a} \mid \mathbf{s})$ to the target values $\mathbb{E}_{\pi(\mathbf{a} \mid \mathbf{s})}[Q(\mathbf{s}, \mathbf{a}) + b(\mathbf{s}, \mathbf{a})]$ to enforce a maximum entropy regularizer on the policy. Alternatively, we could impose a similar form of entropy regularization by adding the bonus directly to the reward. In expectation, we would optimize

$$\begin{aligned}\mathbb{E}_{s, a \sim p_\pi}[r(\mathbf{s}, \mathbf{a}) + \lambda b(\mathbf{s}, \mathbf{a})] &= \frac{1}{H} J(\pi) - \lambda \mathbb{E}_{s \sim p_\pi} \mathbb{E}_{\pi(\mathbf{a} \mid \mathbf{s})} \log \pi(\mathbf{a} \mid \mathbf{s}) \\ &= \frac{1}{H} J(\pi) + \lambda \mathbb{E}_{s \sim p_\pi} \mathcal{H}[\pi(\mathbf{a} \mid \mathbf{s})].\end{aligned}$$

Thus, entropy regularization can also be implemented by adding the bonus $b(\mathbf{s}, \mathbf{a})$ to rewards. In the following parts, you will show how a similar reward bonus can be used to constrain distribution shift in offline RL.

We wish to learn a Q function and policy π from the offline data $\mathcal{D}$ under some constraint $D(\pi, \pi_\beta) \leq \varepsilon$ with the following update:

$$Q(\mathbf{s}, \mathbf{a}) \leftarrow r(\mathbf{s}, \mathbf{a}) + \mathbb{E}_{\mathbf{a}' \sim \pi} [Q(\mathbf{s}', \mathbf{a}')] \quad (1)$$

$$\text{where } \pi = \arg \max_{\pi} \mathbb{E}_{\mathbf{s}, \mathbf{a} \sim p_\pi} [Q(\mathbf{s}, \mathbf{a})] \text{ s.t. } D(\pi, \pi_\beta) \leq \varepsilon. \quad (2)$$

Directly enforcing the constraint in (2) is challenging with the environment rewards $r(\mathbf{s}, \mathbf{a})$, so we will implicitly enforce the constraint with a Lagrangian, modifying the reward to $\bar{r}(\mathbf{s}, \mathbf{a}) = r(\mathbf{s}, \mathbf{a}) + \lambda b(\mathbf{s}, \mathbf{a})$ in (1). The overall optimization then becomes:

$$Q(\mathbf{s}, \mathbf{a}) \leftarrow \bar{r}(\mathbf{s}, \mathbf{a}) + \mathbb{E}_{\mathbf{a}' \sim \pi} [Q(\mathbf{s}', \mathbf{a}')] \quad (3)$$

$$\text{where } \pi = \arg \max_{\pi} \mathbb{E}_{\mathbf{s}, \mathbf{a} \sim p_\pi} [Q(\mathbf{s}, \mathbf{a})]. \quad (4)$$

You may assume that $\lambda > 0$ is selected appropriately to enforce the constraint as follows:

$$\left(\arg \max_{\pi} \mathbb{E}_{\mathbf{s}, \mathbf{a} \sim p_\pi} [Q(\mathbf{s}, \mathbf{a})] - \lambda D(\pi, \pi_\beta) \right) = \left(\arg \max_{\pi} \mathbb{E}_{\mathbf{s}, \mathbf{a} \sim p_\pi} [Q(\mathbf{s}, \mathbf{a})] \text{ s.t. } D(\pi, \pi_\beta) \leq \varepsilon \right).$$

You may also assume access to the distributions $\pi(\mathbf{a} \mid \mathbf{s})$ and $\pi_\beta(\mathbf{a} \mid \mathbf{s})$ in your answers.

1. Suppose we wish to learn π under a KL-divergence constraint, i.e.,

$$D(\pi, \pi_\beta) = \mathbb{E}_{\mathbf{s} \sim p_\pi} D_{KL}[\pi(\mathbf{a} \mid \mathbf{s}) \parallel \pi_\beta(\mathbf{a} \mid \mathbf{s})].$$

How can we enforce this constraint by adding a bonus $b(\mathbf{s}, \mathbf{a})$ to the reward $\bar{r}(\mathbf{s}, \mathbf{a}) = r(\mathbf{s}, \mathbf{a}) + \lambda b(\mathbf{s}, \mathbf{a})$? Write your answer as an expression for $b(\mathbf{s}, \mathbf{a})$.

2. The f -divergence is a generalization of the KL-divergence that can be defined for distributions P and Q by

$$D_f[P \parallel Q] = \int Q(x) f\left(\frac{P(x)}{Q(x)}\right) dx$$

where f is a convex function with zero at 1. We can state an f -divergence policy constraint as

$$\begin{aligned}D(\pi, \pi_\beta) &= \mathbb{E}_{\mathbf{s} \sim p_\pi} D_f[\pi(\mathbf{a} \mid \mathbf{s}) \parallel \pi_\beta(\mathbf{a} \mid \mathbf{s})] \\ &= \mathbb{E}_{\mathbf{s} \sim p_\pi} \mathbb{E}_{\pi_\beta(\mathbf{a} \mid \mathbf{s})} f\left(\frac{\pi(\mathbf{a} \mid \mathbf{s})}{\pi_\beta(\mathbf{a} \mid \mathbf{s})}\right).\end{aligned}$$

作业

5: 探索策略与离线强化学习

g

截止日期: 11月27日星期一, 晚上11:59

1 分析

在本节中, 我们将分析如何使用奖励加成来管理离线强化学习中的分布偏移。我们考虑一个 MDP $M = (\mathcal{S}, \mathcal{A}, r, p)$, 其离线数据 $\mathcal{D}$ 由策略 π_β 收集。我们记 p_π 为由 π 诱导的状态分布。

作为一个激励性示例, 考虑作业 3 中讨论的软演员-评论家 (SAC) 算法。在更新时, SAC 会增加一个调整项 $b(\mathbf{s}, \mathbf{a}) = -\log \pi(\mathbf{a} \mid \mathbf{s})$ 到目标值 $\mathbb{E}_{\pi(\mathbf{a} \mid \mathbf{s})}[Q(\mathbf{s}, \mathbf{a}) + b(\mathbf{s}, \mathbf{a})]$, 以对策略施加最大熵正则化。或者, 我们可以通过直接将奖励加成添加到奖励中来施加类似形式的熵正则化。在期望上, 我们将优化

$$\begin{aligned}\mathbb{E}_{s, a \sim p_\pi}[r(\mathbf{s}, \mathbf{a}) + \lambda b(\mathbf{s}, \mathbf{a})] &= \frac{1}{H} J(\pi) - \lambda \mathbb{E}_{s \sim p_\pi} \mathbb{E}_{\pi(\mathbf{a} \mid \mathbf{s})} \log \pi(\mathbf{a} \mid \mathbf{s}) \\ &= \frac{1}{H} J(\pi) + \lambda \mathbb{E}_{s \sim p_\pi} \mathcal{H}[\pi(\mathbf{a} \mid \mathbf{s})].\end{aligned}$$

因此, 熵正则化也可以通过将奖励加成 $b(\mathbf{s}, \mathbf{a})$ 添加到奖励中来实现。在接下来的部分中, 你将展示如何使用类似的奖励加成来约束离线强化学习中的分布偏移。

我们希望从离线数据 $\mathcal{D}$ 中, 在某个约束 $D(\pi, \pi_\beta) \leq \varepsilon$ 下, 学习一个 Q 函数和策略 π , 更新方式如下:

$$Q(\mathbf{s}, \mathbf{a}) \leftarrow r(\mathbf{s}, \mathbf{a}) + \mathbb{E}_{\mathbf{a}' \sim \pi} [Q(\mathbf{s}', \mathbf{a}')] \quad (1)$$

$$\text{where } \pi = \arg \max_{\pi} \mathbb{E}_{\mathbf{s}, \mathbf{a} \sim p_\pi} [Q(\mathbf{s}, \mathbf{a})] \text{ s.t. } D(\pi, \pi_\beta) \leq \varepsilon. \quad (2)$$

在 (2) 中直接施加约束, 由于环境奖励 $r(\mathbf{s}, \mathbf{a})$ 而具有挑战性, 因此我们将通过拉格朗日乘子隐式施加约束, 将奖励修改为 $\bar{r}(\mathbf{s}, \mathbf{a}) = r(\mathbf{s}, \mathbf{a}) + \lambda b(\mathbf{s}, \mathbf{a})$ 在 (1) 中。整体优化变为:

$$Q(\mathbf{s}, \mathbf{a}) \leftarrow \bar{r}(\mathbf{s}, \mathbf{a}) + \mathbb{E}_{\mathbf{a}' \sim \pi} [Q(\mathbf{s}', \mathbf{a}')] \quad (3)$$

$$\text{where } \pi = \arg \max_{\pi} \mathbb{E}_{\mathbf{s}, \mathbf{a} \sim p_\pi} [Q(\mathbf{s}, \mathbf{a})]. \quad (4)$$

您可以假设 $\lambda > 0$ 被适当选择以施加如下约束:

s:

$$\left(\arg \max_{\pi} \mathbb{E}_{\mathbf{s}, \mathbf{a} \sim p_\pi} [Q(\mathbf{s}, \mathbf{a})] - \lambda D(\pi, \pi_\beta) \right) = \left(\arg \max_{\pi} \mathbb{E}_{\mathbf{s}, \mathbf{a} \sim p_\pi} [Q(\mathbf{s}, \mathbf{a})] \text{ s.t. } D(\pi, \pi_\beta) \leq \varepsilon \right).$$

您还可以假设在回答中可以访问分布 $\pi(\mathbf{a} \mid \mathbf{s})$ 和 $\pi_\beta(\mathbf{a} \mid \mathbf{s})$ 。

1. 假设我们希望在一个KL散度约束下学习 π , 即:

$$D(\pi, \pi_\beta) = \mathbb{E}_{\mathbf{s} \sim p_\pi} D_{KL}[\pi(\mathbf{a} \mid \mathbf{s}) \parallel \pi_\beta(\mathbf{a} \mid \mathbf{s})].$$

我们如何通过向奖励 $\bar{r}(\mathbf{s}, \mathbf{a}) = r(\mathbf{s}, \mathbf{a}) + \lambda b(\mathbf{s}, \mathbf{a})$ 添加一个红利 $b(\mathbf{s}, \mathbf{a})$ 来施加这个约束? 请将你的答案写成 $b(\mathbf{s}, \mathbf{a})$ 的表达式。

2. f -散度是KL散度的一种推广, 可针对分布 P 和 Q 由

$$D_f[P \parallel Q] = \int Q(x) f\left(\frac{P(x)}{Q(x)}\right) dx$$

其中 f 是一个凸函数, 在1处为零。我们可以陈述一个 f -散度策略约束

s

$$\begin{aligned}D(\pi, \pi_\beta) &= \mathbb{E}_{\mathbf{s} \sim p_\pi} D_f[\pi(\mathbf{a} \mid \mathbf{s}) \parallel \pi_\beta(\mathbf{a} \mid \mathbf{s})] \\ &= \mathbb{E}_{\mathbf{s} \sim p_\pi} \mathbb{E}_{\pi_\beta(\mathbf{a} \mid \mathbf{s})} f\left(\frac{\pi(\mathbf{a} \mid \mathbf{s})}{\pi_\beta(\mathbf{a} \mid \mathbf{s})}\right).\end{aligned}$$

The f -divergence constraint allows us to specify many alternative constraints on the divergence between π and π_β . For example, by taking $f(x) = \frac{1}{2}|x-1|$, the f -divergence becomes equivalent to total variation distance (TVD), and by taking $f(x) = -(x+1)\log(\frac{x+1}{2}) + x\log x$ it reduces to the Jensen-Shannon divergence (JSD). When $f(x) = x\log x$ we recover the standard KL divergence.

How can you extend your answer from part (1) to account for an arbitrary f -divergence? Your answer should be a more general alternate expression for $b(\mathbf{s}, \mathbf{a})$ in terms of f .

3. Suppose M has a finite horizon H and we want to constrain divergence in the distribution of *trajectories of states* under π and π_β . We can express the KL divergence between the (state) trajectory distributions for $\tau = (\mathbf{s}_1, \mathbf{s}_2, \dots, \mathbf{s}_H)$ as follows:

$$D(\pi, \pi_\beta) = D_{KL}[p_\pi(\tau) \| p_{\pi_\beta}(\tau)].$$

What expression for $b(\mathbf{s}, \mathbf{a})$ enforces this constraint? You may assume access to the dynamics $p(\mathbf{s}' | \mathbf{s}, \mathbf{a})$. [Hint: marginalize over actions to get a bonus $b(\mathbf{s}, \mathbf{a}, \mathbf{s}')$ that depends on $\mathbf{s}'$. How can this be converted to a bonus $b(\mathbf{s}, \mathbf{a})$ that can be used with $r(\mathbf{s}, \mathbf{a})$?]

2 Coding

This section requires you to implement and evaluate a pipeline for exploration and offline learning. You will first implement an exploration method called random network distillation (RND) and collect data using this exploration procedure, then perform offline training on the data collected via RND using conservative Q-learning (CQL), Advantage Weighted Actor Critic (AWAC), and Implicit Q-Learning (IQL). This assignment will be easier to run on a CPU as we will be using gridworld domains of varying difficulties to train our agents.

The questions will require you to perform multiple runs of offline RL training, which can take quite a long time as we ask you to analyze the empirical significance of specific hyperparameters and thus sweep over them. Furthermore, depending on your implementation, you may find it necessary to tweak some of the parameters, such as learning rates or exploration schedules, which can also be very time consuming. We would **highly** recommend starting early on the coding to allocate enough time to finish the assignment effectively.

2.1 File overview

The starter code for this assignment can be found at

https://github.com/berkeleydeeprlcourse/homework_fall2023/tree/master/hw5

All files needed to run your code are in the **hw5** folder. You will be writing new code in the following files (all in the **hw5/cs285/agents** folder):

- `random_agent.py`
- `rnd_agent.py`
- `dqn_agent.py`
- `cql_agent.py`
- `awac_agent.py`
- `iql_agent.py`

2.2 Environments

Unlike previous assignments, we will consider some stochastic dynamics, discrete-action gridworld environments in this assignment. The three gridworld environments you will need for the graded part of this assignment are of varying difficulty: **easy**, **medium** and **hard**. A picture of these environments is shown below. The easy environment requires following two hallways with a right turn in the middle. The medium environment is a maze requiring multiple turns. The hard environment is a four-rooms task which requires navigating

f -散度约束允许我们对 π 与 π_β 之间的散度施加多种替代约束。例如，取 $f(x) = \frac{1}{2}|x-1|$ ，则 f -散度等价于总变差距离（TVD）；取 $f(x) = -(x+1)\log(\frac{x+1}{2}) + x\log x$ 时，它退化为詹森-香农散度（JSD）。而当 $f(x) = x\log x$ 时，我们恢复标准的KL散度。

你如何将你的答案从部分（1）来考虑任意一个 f 散度？你的答案应该是一个更通用的替代表达式，用于 $b(\mathbf{s}, \mathbf{a})$ 用 f 。

3. 假设 M 有一个有限时域 H ，并且我们希望约束分布中的散度状态轨迹在 π 和 π_β 下。我们可以将对于 $\tau = (\mathbf{s}_1, \mathbf{s}_2, \dots, \mathbf{s}_H)$ 之间的KL散度表示为如下：

$$D(\pi, \pi_\beta) = D_{KL}[p_\pi(\tau) \| p_{\pi_\beta}(\tau)].$$

什么表达式用于 $b(\mathbf{s}, \mathbf{a})$ 强制执行这个约束？你可以假设可以访问动力学 $p(\mathbf{s}' | \mathbf{s}, \mathbf{a})$ 。[提示：对动作进行边缘化以得到一个奖励项 $b(\mathbf{s}, \mathbf{a}, \mathbf{s}')$ 该奖励项依赖于 $\mathbf{s}'$ 。如何将其转化为一个奖励项 $b(\mathbf{s}, \mathbf{a})$ 该奖励项可以与 $r(\mathbf{s}, \mathbf{a})$ ？]

2 编码部分

本节要求你实现并评估一个用于探索与离线学习的流程。首先，你需要实现一种名为随机网络蒸馏（RND）的探索方法，并利用该探索过程收集数据；然后，使用保守Q学习（CQL）、优势加权Actor-Critic（AWAC）和隐式Q学习（IQL）对通过RND收集的数据进行离线训练。本次作业更易于在CPU上运行，因为我们将在不同难度的网格世界环境中训练智能体。

问题部分要求你进行多次离线强化学习训练，这可能会耗费相当长的时间，因为我们需要你分析特定超参数的实证意义，并因此对这些参数进行扫描。此外，根据你的实现，你可能会发现有必要调整一些参数（如学习率或探索计划），这同样非常耗时。我们**强烈**建议尽早开始编码部分，以便为有效完成作业留出充足时间。

2.1 文件概览

本作业的起始代码可在以下网址找到：

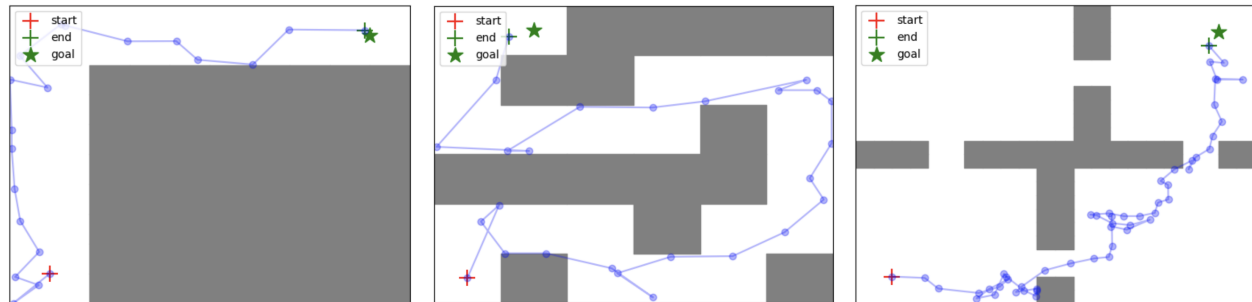
https://github.com/berkeleydeeprlcourse/homework_fall2023/tree/master/hw5

运行代码所需的所有文件都在hw5文件夹中。你将在以下文件中编写新代码（全部位于hw5/cs285/agents文件夹中）：

- 随机智能体.py
- rnd智能体.py
- dqn 智能体.py
- cql 智能体.py
- awac 智能体.py
- iql 智能体.py

2.2 环境

与之前的作业不同，在本作业中，我们将考虑一些随机动力学、离散动作的网格世界环境。本次作业评分部分所需的三个网格世界环境难度各异：简单、中等和困难。这些环境的图片如下所示。简单环境需要沿着两条走廊直行并在中间右转。中等环境是一个需要多次转弯的迷宫。困难环境是一个四房间任务，需要穿越

Figure 1: Figures depicting the **easy** (left), **medium** (middle) and **hard** (right) environments.

between multiple rooms through narrow passages to reach the goal location. We also provide a very hard environment for the bonus (optional) part of this assignment.

2.3 Random Network Distillation (RND) Algorithm

A common way of doing exploration is to visit states with a large prediction error of some quantity, for instance, the TD error or even random functions. The RND algorithm, as covered in Lecture 13, aims at encouraging exploration by asking the exploration policy to more frequently undertake transitions where the prediction error of a random neural network function is high. Formally, let $f_{\theta}^*(s')$ be a randomly chosen vector-valued function represented by a neural network. RND trains another neural network, $\hat{f}_{\phi}(s')$ to match the predictions of $f_{\theta}^*(s')$ under the distribution of datapoints in the buffer, as shown below:

$$\phi^* = \arg \min_{\phi} \mathbb{E}_{s,a,s' \sim \mathcal{D}} \left[\underbrace{\|\hat{f}_{\phi}(s') - f_{\theta}^*(s')\|}_{\mathcal{E}_{\phi}(s')} \right]. \quad (5)$$

If a transition (s, a, s') is in the distribution of the data buffer, the prediction error $\mathcal{E}_{\phi}(s')$ is expected to be small. On the other hand, for all unseen state-action tuples it is expected to be large. To utilize this prediction error as a reward bonus for exploration, RND trains two critics – an *exploitation critic*, $Q_R(s, a)$, and an *exploration critic*, $Q_{\mathcal{E}}(s, a)$, where the exploitation critic estimates the return of the policy under the actual reward function and the exploration critic estimates the return of the policy under the reward bonus. In practice, we normalize error before passing it into the exploration critic, as this value can vary widely in magnitude across states leading to poor optimization dynamics.

In this problem, we represent the random functions utilized by RND, $f_{\theta}^*(s')$ and $\hat{f}_{\phi}(s')$ via random neural networks. To prevent the neural networks from having zero prediction error right from the beginning, we initialize the target f_{θ}^* using an alternative initialization scheme in `agents/rnd_agent.py`.

2.4 Conservative Q-Learning (CQL) Algorithm

For the first portion of the offline RL part of this assignment, we will implement the conservative Q-learning (CQL) algorithm. The goal of CQL is to preventing overestimation of the policy value. In order to do that, a conservative, lower-bound Q-function is learned by additionally minimizing Q-values alongside a standard Bellman error objective. This is done by augmenting the Q-function training with a regularizer that minimizes the soft-maximum of the Q-values $\log(\sum_a \exp(Q(s, a)))$ and maximizes the Q-value on the state-action pair seen in the dataset, $Q(s, a)$. The overall CQL objective is given by the standard TD error objective augmented with the CQL regularizer weighted by α : $\alpha \left[\frac{1}{N} \sum_{i=1}^N (\log(\sum_a \exp(Q(s_i, a))) - Q(s_i, a_i)) \right]$. You will tweak this value of α in later questions in this assignment.

2.5 Advantage Weighted Actor Critic (AWAC) Algorithm

For the second portion of the offline RL part of this assignment, we will implement the AWAC algorithm. This augments the training of the policy by utilizing the following actor update:

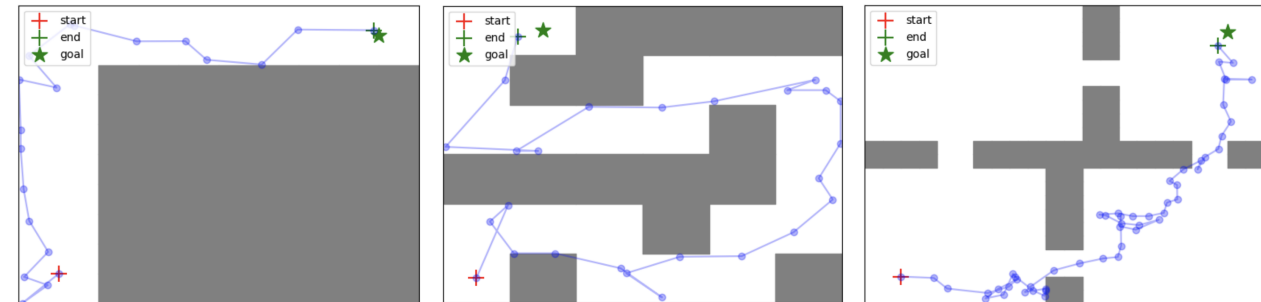


Figure 1: 描绘简单（左）、中等（中）和困难（右）环境的图像

s.

多个房间之间的狭窄通道才能到达目标位置。我们还为本次作业的加分（可选）部分提供了一个非常困难的环境。

2.3 随机网络蒸馏（RND）算法

探索的一种常见方法是访问某些量的预测误差较大的状态，例如TD误差甚至随机函数。如第13讲所述，RND算法旨在通过让探索策略更频繁地执行那些随机神经网络函数预测误差较高的转移来鼓励探索。形式上，设 $f_{\theta}^*(s')$ 是一个由神经网络表示的随机选择的向量值函数。RND训练另一个神经网络 $\hat{f}_{\phi}(s')$ ，使其在缓冲区中数据点的分布下匹配 $f_{\theta}^*(s')$ 的预测，如下所示：

$$\phi^* = \arg \min_{\phi} \mathbb{E}_{s,a,s' \sim \mathcal{D}} \left[\underbrace{\|\hat{f}_{\phi}(s') - f_{\theta}^*(s')\|}_{\mathcal{E}_{\phi}(s')} \right]. \quad (5)$$

如果一个转移 (s, a, s') 位于数据缓冲区的分布中，预测误差 $\mathcal{E}_{\phi}(s')$ 预计会很小。另一方面，对于所有未见过的状态-动作元组，它预计会很大。为了利用这个预测误差作为探索的奖励加成，RND训练了两个评估网络——一个利用评估网络， $Q_R(s, a)$ ，和一个探索评估网络， $Q_{\mathcal{E}}(s, a)$ ，其中利用评估网络估计策略在实际奖励函数下的回报，探索评估网络估计策略在奖励加成下的回报。在实践中，我们会在将误差传入探索评估网络之前对其进行归一化，因为该值在不同状态之间的幅度可能变化很大，导致优化动态不佳。

在这个问题中，我们通过随机神经网络来表示 RND 所使用的随机函数， $f_{\theta}^*(s')$ 和 $\hat{f}_{\phi}(s')$ 。为了防止神经网络从一开始就具有零预测误差，我们使用 `agents/rnd_agent.py` 中的替代初始化方案来初始化目标 f_{θ}^* 使用。

2.4 保守Q学习（CQL）算法

对于本作业离线强化学习部分的第一部分，我们将实现保守Q学习（CQL）算法。CQL的目标是防止对策略价值的高估。为此，通过额外最小化Q值并结合标准贝尔曼误差目标，学习一个保守的下界Q函数。具体做法是：在Q函数训练中增加一个正则化项，该项最小化Q值的soft-maximum对数 $\log(\sum_a \exp(Q(s, a)))$ ，并最大化数据集中观察到的状态-动作对的Q值 $Q(s, a)$ 。总体CQL目标由标准TD误差目标加上由 α 加权的CQL正则化项组成： $\alpha \left[\frac{1}{N} \sum_{i=1}^N (\log(\sum_a \exp(Q(s_i, a))) - Q(s_i, a_i)) \right]$ 。在后来的作业问题中，你将调整此 α 的值。

2.5 优势加权演员-评论家 (AWAC) 算法

在本作业的离线强化学习部分的第二部分，我们将实现AWAC算法。该算法通过使用以下演员更新来增强策略的训练：

$$\theta \leftarrow \arg \max_{\theta} \mathbb{E}_{s,a \sim \mathcal{B}} \left[\log \pi_{\theta}(a|s) \exp\left(\frac{1}{\lambda} \mathcal{A}^{\pi_k}(s,a)\right) \right]. \quad (6)$$

This update is similar to weighted behavior cloning (which it resolves to if the Q function is degenerate). But with a well-formed Q estimate, we weight the policy towards selecting actions that are high under our learnt q function. In the update above, the agent regresses onto high-advantage actions with a large weight, while almost ignoring low-advantage actions. This actor update amounts to weighted maximum likelihood (i.e., supervised learning), where the targets are obtained by re-weighting the state-action pairs observed in the current dataset by the predicted advantages from the learned critic, without explicitly learning any parametric behavior model, simply sampling (s, a) from the replay buffer β .

The Q function is learnt with a Temporal Difference (TD) Loss. The objective can be found below.

$$\mathbb{E}_D[(Q(s,a) - r(s,a) + \gamma \mathbb{E}_{a' \sim \pi}[Q_{\phi_{k-1}}(s',a')])^2] \quad (7)$$

Note that next actions a' are sampled from the learned policy π , meaning that OOD actions will not be sampled if π does a good job of fitting the (weighted) behavior policy.

2.6 Implicit Q-Learning (IQL) Algorithm

For the second portion of the offline RL part of this assignment, we will implement the IQL algorithm. IQL decouples the problem of learning the critic from the policy learning by using an *implicit* Bellman backup rather than explicitly considering the backup under a particular policy. It does this by learning an *expectile* of Q , which is a statistic similar to a quantile. This is a “soft” version of the *maximum* value attained by a distribution. For a random variable X , the expectile $m_{\tau}(X)$ is given as to minimize the following:

$$\arg \min_{m_{\tau}} \mathbb{E}_{x \sim X} [L_2(x - m_{\tau})], L_2^{\tau}(\mu) = |\tau - \mathbb{1}\{\mu \leq 0\}| \mu^2$$

This backup will act optimistically with respect to actions taken in the dataset. To avoid being optimistic to *state* transitions, we need to learn a separate value function $V(s)$ that performs the optimism, then regress $Q(s,a) \leftarrow r + \gamma V(s')$ with a regular MSE loss. All together:

$$L_V(\phi) = \mathbb{E}_{(s,a) \sim D} [L_2^{\tau}(Q_{\theta}(s,a) - V_{\phi}(s))] \quad (8)$$

$$L_Q(\theta) = \mathbb{E}_{(s,a,s') \sim D} [(r(s,a) + \gamma V_{\phi}(s') - Q_{\theta}(s,a))^2] \quad (9)$$

Note that the critic learning process has two nice properties:

1. It never queries out-of-distribution actions (e.g. actions a' from an arbitrary policy), which completely avoids the OOD overestimation issue.
2. It can be conducted without an actor update, so if you want you can first train a critic and then only train the actor at the end.

The actor update is decoupled from the critic update (hence *implicit* Q-learning), and is learned with the same objective as AWAC:

$$L_{\pi}(\psi) = -\mathbb{E}_{s,a \sim \mathcal{B}} \left[\log \pi_{\psi}(a|s) \exp\left(\frac{1}{\lambda} \mathcal{A}^{\pi_k}(s,a)\right) \right]. \quad (10)$$

2.7 Relevant Literature

For more details about the algorithmic implementation, feel free to refer to the following papers: [Conservative Q-Learning for Offline Reinforcement Learning \(CQL\)](#), [Accelerating Online Reinforcement Learning with Offline Datasets \(AWAC\)](#), [Offline Reinforcement Learning with Implicit Q-Learning \(IQL\)](#), and [Exploration by Random Network Distillation \(RND\)](#).

$$\theta \leftarrow \arg \max_{\theta} \mathbb{E}_{s,a \sim \mathcal{B}} \left[\log \pi_{\theta}(a|s) \exp\left(\frac{1}{\lambda} \mathcal{A}^{\pi_k}(s,a)\right) \right]. \quad (6)$$

这个更新类似于加权行为克隆（如果Q函数是退化的，它会简化为加权行为克隆）。但是，在具有良好估计的Q值的情况下，我们会根据学习到的Q函数，将策略权重偏向于选择高价值的动作。在上述更新中，代理以较大的权重回归到高优势动作，而几乎忽略低优势动作。这个演员更新相当于加权最大似然估计（即监督学习），其中目标是通过将当前数据集中观察到的状态-动作对重新加权来获得的，权重来自学习到的评论家预测的优势，而不显式学习任何参数化行为模型，只是从回放缓冲区 β 中采样 (s, a)。

Q函数通过时序差分（TD）损失进行学习。目标函数如下所示。

$$\mathbb{E}_D[(Q(s,a) - r(s,a) + \gamma \mathbb{E}_{a' \sim \pi}[Q_{\phi_{k-1}}(s',a')])^2] \quad (7)$$

注意，后续动作 a' 是从学习到的策略 π 中采样的，这意味着如果 π 能够很好地拟合（加权的）行为策略，就不会采样到分布外动作。

2.6 隐式Q学习 (IQL) 算法

对于本次作业中离线RL部分的第二部分，我们将实现IQL算法。IQL通过使用隐式贝尔曼备份，而不是显式考虑在特定策略下的备份，将评论家的学习问题与策略学习解耦。它通过学习 Q 的一个期望分位数来实现这一点，这是一种类似于分位数的统计量。这是分布所达到的最大值的“软”版本。对于随机变量 X ，期望分位数 $m_{\tau}(X)$ 通过最小化以下式子给出：

$$\arg \min_{m_{\tau}} \mathbb{E}_{x \sim X} [L_2(x - m_{\tau})], L_2^{\tau}(\mu) = |\tau - \mathbb{1}\{\mu \leq 0\}| \mu^2$$

这种备份将对数据集中采取的动作持乐观态度。为了避免对状态转移持乐观态度，我们需要学习一个单独的函数 $V(s)$ 来执行这种乐观估计，然后使用常规的MSE损失对 $Q(s,a) \leftarrow r + \gamma V(s)$ 进行回归。总结如下：

$$L_V(\phi) = \mathbb{E}_{(s,a) \sim D} [L_2^{\tau}(Q_{\theta}(s,a) - V_{\phi}(s))] \quad (8)$$

$$L_Q(\theta) = \mathbb{E}_{(s,a,s') \sim D} [(r(s,a) + \gamma V_{\phi}(s') - Q_{\theta}(s,a))^2] \quad (9)$$

注意，评论家的学习过程有两个很好的特性：

1. 它从不查询分布外的动作（例如来自任意策略的动作 a' ），这完全避免了OOD高估问题。
2. 它可以在不进行演员更新的情况下进行，因此如果你愿意，可以先训练评论家，最后再训练演员。

演员更新与评论家更新解耦（因此称为隐式 Q -学习），并且使用与AWAC相同的目标进行学习：

$$L_{\pi}(\psi) = -\mathbb{E}_{s,a \sim \mathcal{B}} \left[\log \pi_{\psi}(a|s) \exp\left(\frac{1}{\lambda} \mathcal{A}^{\pi_k}(s,a)\right) \right]. \quad (10)$$

2.7 相关文献

关于算法实现的更多细节，欢迎参考以下论文：[用于离线强化学习的保守Q学习（CQL）](#)，[利用离线数据集加速在线强化学习（AWAC）](#)，[基于隐式Q学习的离线强化学习（IQL）](#)，以及[基于随机网络蒸馏的探索（RND）](#)。

2.8 Implementation

The first part in this assignment is to implement a working version of Random Network Distillation. The default code will run the **easy** environment with reasonable hyperparameter settings. Look for the **# TODO(student)** markers in the files listed above for detailed implementation instructions.

2.9 Evaluation

Once you have a working implementation of RND, CQL, AWAC, and IQL, you should prepare a report. The report should consist of one figure for each question below (each part has multiple questions). You should turn in the report as one PDF and a zip file with your code. If your code requires special instructions or dependencies to run, please include these in a file called **README** inside the zip file.

3 Exploration

In RL, our agent needs to see high-reward transitions at some point during training to understand that they exist. Your previous assignments (PG, DQN, SAC) have just used random exploration, possibly with some state-dependent noise (like in SAC). Here, you'll instead implement a policy that explicitly maximizes state coverage to explore the entire space.

Later, we'll use data collected from these exploration policies with several different offline RL algorithms.

3.1 Running a random policy

Just to get a sense for the three environments, implement **get_action** for the **RandomAgent** in **random_agent.py**. Run the random policy to generate random exploration in each of the three environments:

```
python cs285/scripts/run_hw5_explore.py \
  -cfg experiments/exploration/pointmass_easy_random.yaml
  --dataset_dir datasets/
python cs285/scripts/run_hw5_explore.py \
  -cfg experiments/exploration/pointmass_medium_random.yaml \
  --dataset_dir datasets/
python cs285/scripts/run_hw5_explore.py \
  -cfg experiments/exploration/pointmass_hard_random.yaml \
  --dataset_dir datasets/
```

These scripts will save visualizations of the final dataset in the **exploration** directory, as well as intermediate results in the Tensorboard logs. Include the final dataset visualization in your report.

3.2 Random Network Distillation

What you will implement: the RND algorithm for exploration. You will need to change **cs285/agents/rnd_agent.py**. In addition, you should also thoroughly read through the training scheme in **run_hw5_explore.py**. It's very similar to the DQN training scheme you implemented in HW3.

Implement the RND algorithm and use the argmax policy with respect to the exploration critic to generate state-action tuples to populate the replay buffer for the algorithm. In the code, this happens before the number of iterations crosses **num_exploration_steps**, which is set to 10k by default. You need to collect data using the **ArgmaxPolicy** policy which chooses to perform actions that maximize the exploration critic value.

The experiment logs contain visualizations of RND error computed at each point in the environment as well as a scatter plot of visited states. As exploration progresses, a working RND algorithm should accumulate low error in all reachable states and high error in unreachable states (e.g. walls).

First, make sure your RND implementation works in the easy environment. Then, run it in all three environments:

2.8 实现

本次作业的第一部分是实现一个可运行的随机网络蒸馏版本。默认代码使用合理的超参数设置运行简单环境。请在上文列出的文件中查找 **# TODO(student)** 标记，以获取详细的实现说明。

2.9 评估

当你拥有 RND、CQL、AWAC 和 IQL 的有效实现后，应准备一份报告。报告应包含以下每个问题对应的图表（每个部分有多多个问题）。你需要提交一份 PDF 格式的 report 和一个包含代码的压缩包。如果你的代码需要特殊的运行说明或依赖项，请将这些内容包含在压缩包内的 README 文件中。

3 探索

在强化学习中，我们的智能体需要在训练过程中的某个时刻看到高奖励的转移，才能理解它们的存在。你之前的作业（PG、DQN、SAC）仅使用了随机探索，可能还带有一些状态依赖的噪声（如SAC中那样）。而在这里，你将实现一种策略，显式地最大化状态覆盖范围，从而探索整个空间。

之后，我们将使用这些探索策略收集的数据，配合几种不同的离线强化学习算法进行处理。

3.1 运行随机策略

为了初步了解这三个环境，请在 **random_agent.py** 中为 **RandomAgent** 实现 **get_action** 方法。运行随机策略，在三个环境中分别生成随机探索数据：

```
python cs285/scripts/run_hw5_explore.py \ -cfg
```

这些脚本将把最终数据集的可视化结果保存到探索目录中，同时将中间结果保存到Tensorboard日志中。请将最终数据集的可视化结果包含在报告中。

3.2 随机网络蒸馏

你将实现的内容：用于探索的RND算法。你需要修改**cs285/agents/rnd_agent.py**。此外，你还需要仔细阅读**run_hw5_explore.py**中的训练方案。这与你在HW3中实现的DQN训练方案非常相似。

实现RND算法，并使用相对于探索评论家的argmax策略生成状态-动作元组，以填充算法的回放缓冲区。在代码中，这发生在迭代次数超过**num_exploration_steps**之前，默认设置为10k。你需要使用**ArgmaxPolicy**策略收集数据，该策略选择执行最大化探索评论家价值的动作。

实验日志包含在环境中每个点计算的RND误差的可视化，以及访问状态的散点图。随着探索的进行，一个有效的RND算法应该在所有可达状态中积累低误差，而在不可达状态（例如墙壁）中积累高误差。

首先，确保你的RND实现在简单环境中能正常工作。然后，在所有三种环境中运行它：

```
python cs285/scripts/run_hw5_explore.py \
    -cfg experiments/exploration/pointmass_easy_rnd.yaml
--dataset_dir datasets/
python cs285/scripts/run_hw5_explore.py \
    -cfg experiments/exploration/pointmass_medium_rnd.yaml \
    --dataset_dir datasets/
python cs285/scripts/run_hw5_explore.py \
    -cfg experiments/exploration/pointmass_hard_rnd.yaml \
    --dataset_dir datasets/
```

Again, include the visualizations in your final report. **These visualizations, particularly the RND error map, will be very helpful for debugging!**

4 Offline RL

4.1 CQL

Now that we have implemented RND for collecting exploration data that is (likely) useful for performing exploitation, we will perform offline RL on this dataset and see how close the resulting policy is to the optimal policy. To begin, you will implement the conservative Q-learning algorithm in this `cs285/agents/cql_agent.py`.

Then, run both a standard DQN agent and your new CQL agent in the offline setting with the datasets you collected earlier.

```
python ./cs285/scripts/run_hw5_offline.py \
    -cfg experiments/offline/pointmass_easy_cql.yaml \
    --dataset_dir datasets
python ./cs285/scripts/run_hw5_offline.py \
    -cfg experiments/offline/pointmass_medium_dqn.yaml \
    --dataset_dir datasets

python ./cs285/scripts/run_hw5_offline.py \
    -cfg experiments/offline/pointmass_easy_cql.yaml \
    --dataset_dir datasets
python ./cs285/scripts/run_hw5_offline.py \
    -cfg experiments/offline/pointmass_medium_dqn.yaml \
    --dataset_dir datasets
```

On the Medium environment, create several experiment variations in which the value of the α parameter is varied, from $\alpha = 0$ (equivalent to DQN) to $\alpha = 10$. Show both resulting Q -values and evaluation performances in a plot. In the caption, describe how the α parameter affects training and performance in offline RL.

4.2 Policy Constraint Methods: IQL and AWAC

While CQL learns an actor via a modification to actor-critic algorithms like DQN that regularizes actions towards those found in the dataset by decreasing OOD Q -values, AWAC learns an in-distribution policy directly by performing weighted behavior cloning on the dataset.

Implement AWAC in `cs285/agents/awac_agent.py`, and IQL in `cs285/agents/iql_agent.py`. Compare them using the IQL and AWAC configuration files in the `experiments` directory. Report evaluation curves for both approaches.

4.3 Data ablations

Finally, compare the performance of offline RL under several different sizes of dataset. Run RND with `total_steps` 1000, 5000, 10000, and 20000 on either the Medium or Hard environment, creating a new

```
python cs285/scripts/run_hw5_explore.py \ -cfg
```

再次强调，请在你的最终报告中包含可视化内容。**这些可视化内容，尤其是RND误差图，将非常有助于调试！**

4 离线强化学习

4.1 CQL

既然我们已经实现了RND来收集（很可能）对利用有用的探索数据，接下来我们将在此数据集上执行离线强化学习，并观察所得策略与最优策略的接近程度。首先，你需要在`cs285/agents/cql_agent.py`中实现保守Q学习算法。

然后，使用你之前收集的数据集，在离线设置中分别运行标准DQN智能体和你的新CQL智能体。

```
python ./cs285/scripts/run_hw5_offline.py \ -cfg
```

在Medium环境中，创建多个实验变体，其中 α 参数的值从 $\alpha = 0$ （相当于DQN）变化到 $\alpha = 10$ 。在图表中展示生成的 Q -值和评估性能。在图注中，描述 α 参数如何影响离线RL中的训练和性能。

4.2 策略约束方法：IQL与AWAC

CQL通过对类似DQN的actor-critic算法进行修改来学习actor，该算法通过降低OOD Q -值来使动作正则化到数据集中找到的动作，而AWAC则通过对数据集执行加权行为克隆来直接学习分布内策略。

在`cs285/agents/awac_agent.py`中实现AWAC，在`cs285/agents/iql_agent.py`中实现IQL。使用`experiments`目录中的IQL和AWAC配置文件对它们进行比较。报告两种方法的评估曲线。

4.3 数据消融实验

最后，比较在不同数据集大小下离线强化学习的表现。在Medium或Hard环境中，使用RND运行总计_步数1000、5000、10000和20000，创建一个新的

dataset for each variation (you will need to make several .yaml config files for this). Then, train a CQL agent on each dataset and report its performance as well as evaluation curves.

5 Online Fine-Tuning

So far we only support training an algorithm purely offline, using data collected in a previous run. In `run_hw5_finetune.py`, implement online fine-tuning by first loading an offline dataset and taking a fixed number of training steps with an offline RL algorithm of your choice (IQL, CQL, or AWAC) and then switching to online learning while keeping all of the data in the dataset to initialize your replay buffer.

Report results as evaluation returns, clearly indicating the point at which online fine-tuning begins. An example configuration is provided for you in `experiments/finetune/pointmass_hard_cql_finetune.yaml`.

With online fine-tuning, your policy should be able to (stably) reach high reward (at least -20) on `PointmassHard-v0`.

6 Submitting the code and experiment runs

In order to turn in your code and experiment logs, create a folder that contains the following:

- A folder named `data` with all the experiment runs from this assignment. **Do not change the names originally assigned to the folders, as specified by `exp_name` in the instructions. Video logging is not utilized in this assignment, as visualizations are provided through plots, which are outputted during training.**
- The `cs285` folder with all the `.py` files, with the same names and directory structure as the original homework repository (excluding the `data` folder). Also include any special instructions we need to run in order to produce each of your figures or tables (e.g. “run python myassignment.py -sec2q1” to generate the result for Section 2 Question 1) in the form of a README file.

If you are a Mac user, **do not use the default “Compress” option to create the zip**. It creates artifacts that the autograder does not like. You may use `zip -vr submit.zip submit -x "*.DS_Store"` from your terminal.

Turn in your assignment on Gradescope. Upload the zip file with your code and log files to **HW5 Code**, and upload the PDF of your report to **HW5**.

As an example, the unzipped version of your submission should result in the following file structure. **Make sure that the submit.zip file is below 15MB and that they include the prefix q1-, q2-, q3-, etc.**

```
submit.zip
├── data
│   ├── q1...
│   │   └── events.out.tfevents.1567529456.e3a096ac8ff4
│   └── ...
├── cs285
│   └── ...
└── ...
```

每个变体的数据集（为此你需要创建多个.yaml配置文件）。然后，在每个数据集上训练一个CQL代理，并报告其性能以及评估曲线。

5 在线微调

到目前为止，我们只支持使用先前运行中收集的数据纯粹离线训练算法。在 `run_hw5_finetune.py` 中，实施在线微调，先加载离线数据集，使用你选择的离线RL算法（IQL、CQL或AWAC）进行固定数量的训练步骤，然后切换到在线学习，同时保留数据集中的所有数据来初始化你的回放缓冲区。

以评估回报的形式报告结果，并清晰地标明在线微调开始的时间点。示例配置已在 `experiments/finetune/pointmass_hard_cql_finetune.yaml` 中为你提供。

通过在线微调，你的策略应当能够（稳定地）在 `PointmassHard-v0` 上达到高奖励（至少 -20）。

6 提交代码和实验运行记录

为了提交你的代码和实验日志，请创建一个包含以下内容的文件夹：

- 一个名为 `data` 的文件夹，内含本次作业的所有实验运行记录。 **不要更改文件夹最初指定的名称，应严格按照说明中 `exp name` 的要求进行命名。本次作业不使用视频记录功能，因为通过训练过程中输出的图表即可提供可视化结果。**
- `cs285` 文件夹，内含所有 `.py` 文件，保持与原始作业仓库相同的文件名和目录结构（不包括 `data` 文件夹）。同时，请以 README 文件的形式附上我们为生成每个图表或表格所需运行的特殊指令（例如，“运行 python myassignment.py -sec2q1”以生成第 2 节问题 1 的结果）。

如果您是Mac用户，**请勿使用默认的“压缩”选项来创建zip文件**。这会产生自动评分器不喜欢的文件。您可以在终端中使用 `zip -vr submit.zip submit -x "*.DS Store"`。

在Gradescope上提交您的作业。将包含代码和日志文件的zip文件上传到**HW5代码**，并将报告的PDF上传到**HW5**。

例如，解压后的提交文件应具有以下文件结构。**确保submit.zip文件小于15MB，并且包含前缀q1, q2, q3, 等。**

```
submit.zip data q1...
events.out.tfevents.1567529456.e3a096ac8ff4 ... cs285 ...
...
├──
└──
```